UNIT - V

INPUT / OUTPUT ORGANIZATION

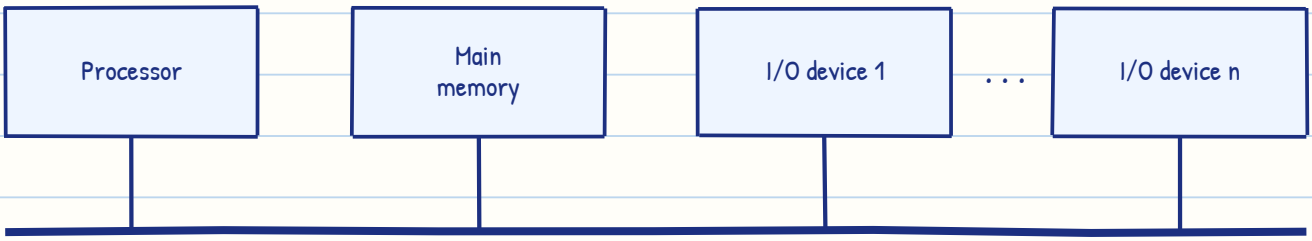
Computer Organization - Handwritten Study Notes

Syllabus topics:

Accessing I/O Devices, Interrupts, Processor Examples, Direct Memory Access, Buses, Interface Circuits, Standard I/O Interfaces

1. Accessing I/O Devices

Input/output (I/O) devices - keyboard, mouse, display, printer, disk, network card - let the computer talk to the outside world. They are connected to the processor and memory through a **bus** .



Single bus: address lines + data lines + control lines

Fig 1: A single-bus structure

- The bus has **address lines, data lines and control lines** . Every I/O device is given an **address** (just like a memory location), so the processor can select it.

Two ways of addressing I/O devices

- **Memory-mapped I/O:** I/O devices and memory share **one address space** . Any instruction that accesses memory (Move, Load, Store) can also read or write a device register. Example: Move DATAIN, R0 reads the keyboard; Move R0, DATAOUT writes to the display. No special instructions are needed.
- **I/O-mapped (isolated) I/O:** I/O devices have a **separate address space** and the processor has special **IN / OUT** instructions. A control line (**IO/M**) tells whether the address is for I/O or memory.

Memory-mapped I/O	Isolated (I/O-mapped) I/O
One address space for memory + I/O	Separate address spaces
Normal Load/Store/Move instructions	Special IN / OUT instructions
Uses up some memory addresses	Full memory address space is free
Used by ARM, most RISC processors	Used by Intel x86 (and also memory-mapped)

I/O interface

The **I/O interface** is a circuit between the bus and the device. The device is much slower and works differently from the processor, so the interface adapts them.

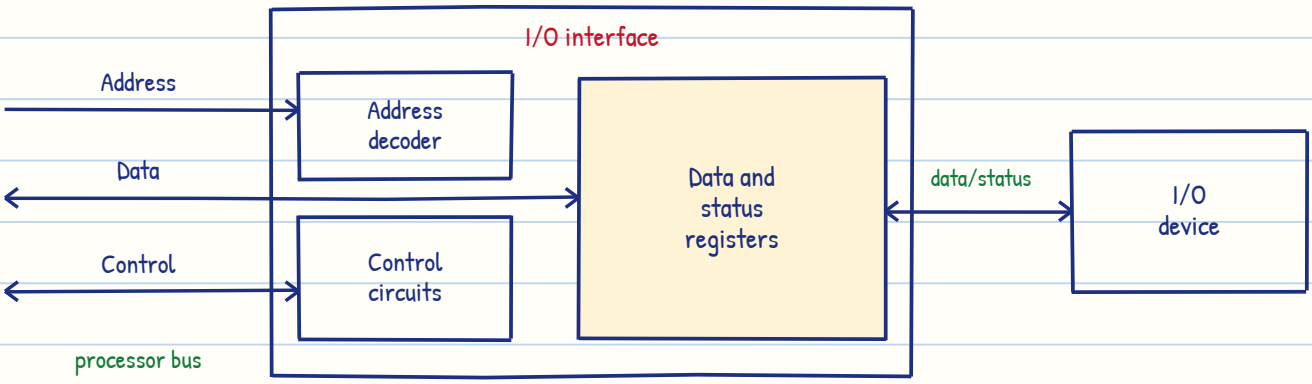


Fig 2: I/O interface for an input/output device

- **Address decoder:** recognises the device address on the address lines.
- **Data register** (e.g. **DATAIN** , **DATAOUT**) holds the data being transferred. **Status register** holds flags such as **SIN** (input data ready) and **SOUT** (output device ready).
- **Control circuits:** handle the timing and control signals.

Program-controlled I/O (polling)

The processor **repeatedly checks (polls)** the status flag until the device is ready, then transfers the data. Example: read one character from the keyboard and echo it on the display.

```
READWAIT: Branch to READWAIT if SIN = 0
          Move DATAIN, R1 ; read the character
WRITEWAIT: Branch to WRITEWAIT if SOUT = 0
          Move R1, DATAOUT ; display the character
```

- **Drawback:** the processor wastes all its time in the waiting loop (busy waiting). Devices are thousands of times slower than the CPU.
- **Solutions:** use **interrupts** (device calls the processor when ready) or **DMA** (block transfer without the processor).

2. Interrupts

Instead of waiting, the processor does useful work. When the device is ready it sends an **interrupt-request** signal on a control line. The processor stops the current program temporarily, runs a special routine, and then continues.

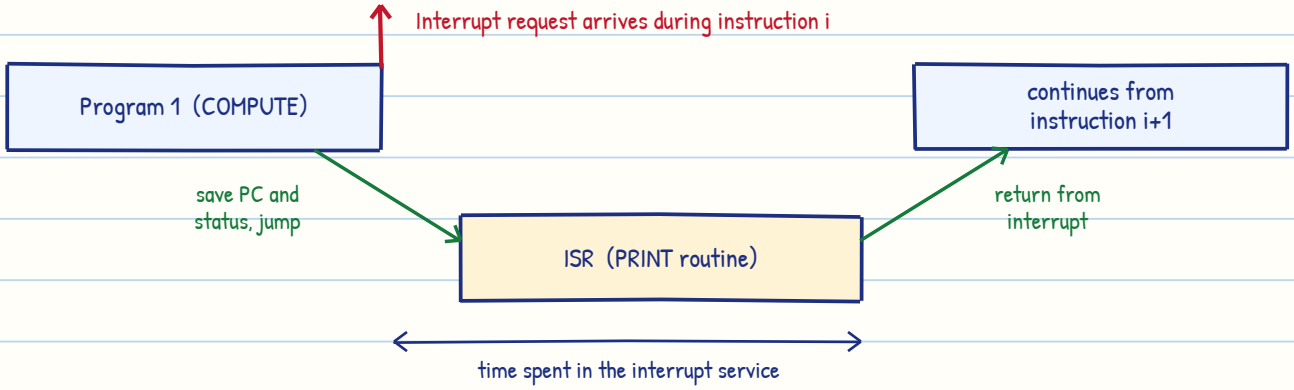


Fig 3: Transfer of control through an interrupt

What happens on an interrupt

1. Processor **completes the current instruction** (instruction i).
2. It **saves the return address (PC)** and the processor status (PS), then **disables further interrupts** (IE = 0).
3. It jumps to the **Interrupt Service Routine (ISR)** for that device. The ISR serves the device and **removes the request** .
4. A **Return-from-interrupt** instruction restores PC and PS and the program continues at instruction i+1.

- **Interrupt latency** = delay between the arrival of the request and the start of the ISR.
- ISR vs subroutine: a subroutine is **called by the program** at a known place; an ISR can run at **any time** (unplanned), so it must save and restore every register it uses, and the interrupted program never knows.

Enabling and disabling interrupts

- **IE bit** in the processor status register: if 0, interrupt requests are ignored (e.g. while an ISR runs or during a critical section).
- Each device interface has its own **interrupt-enable bit** in its control register (e.g. **KEN** for keyboard, **DEN** for display), so a device can be allowed or blocked.
- **Interrupt hardware:** all devices share one request line **INTR** (wired-OR). The processor answers with **INTA** (interrupt acknowledge).

Handling many devices

When several devices share the INTR line the processor must find out **who interrupted** and **who goes first** .

1. **Polling the status registers:** the ISR checks the IRQ bit of every device in turn. Simple, but wastes time.
2. **Vectored interrupts:** the device sends a **vector** (its code, or the starting address of its ISR) on the data lines when INTA arrives. The processor jumps directly to the right ISR - **fast** .
3. **Interrupt nesting:** a high-priority device may interrupt the ISR of a low-priority device. The processor has a **priority level** in PS and accepts a request only if its priority is **higher** than the current one.

Priority and simultaneous requests

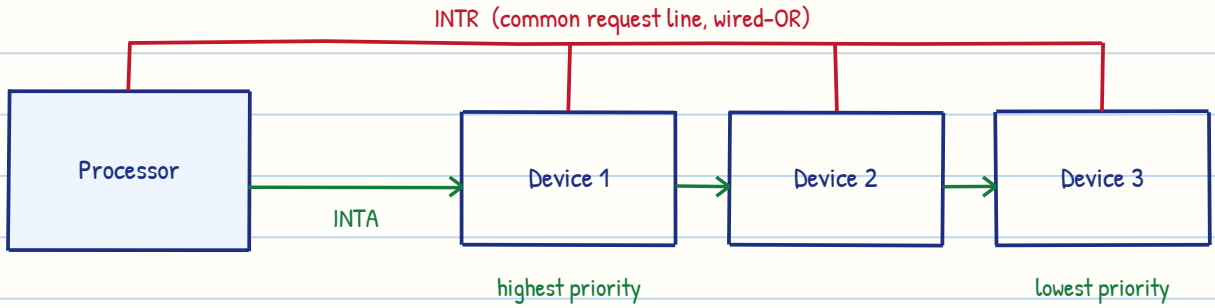


Fig 4: Daisy-chain interrupt priority

- **Daisy chain:** INTA is passed from device to device. A device that has requested keeps INTA and puts its vector; otherwise it passes INTA on. The device **nearest the processor has the highest priority** .
- **Separate INTR/INTA lines** for each device (independent priority) or **groups of daisy chains** are also used, giving flexible priorities.

Exceptions

Exceptions are interrupts generated inside the processor (not by I/O devices) and are handled by similar ISRs.

- **Recovery from errors:** illegal instruction, arithmetic overflow, divide by zero, memory (ECC) error, page fault.
- **Debugging: trace mode** (exception after each instruction) and **breakpoints** (special instruction) let a debugger take control.
- **Privilege exception:** a user program tries to run a privileged instruction (I/O, change of interrupt mask) - only the OS may do it.

3. Processor Examples

The same ideas (device registers, status flags, interrupt vectors) are used by real processors. Two common examples are **ARM** and **Intel IA-32 (Pentium)** .

3.1 ARM

- **I/O is memory-mapped:** device registers are accessed with normal Load/Store instructions at fixed addresses. Parallel-port and serial-port interfaces use data, status and control registers.
- ARM has two interrupt levels: **IRQ** (normal) and **FIQ** (fast interrupt, higher priority).
- **CPSR** (current program status register) has the **I bit** (1 = IRQ disabled), the **F bit** (1 = FIQ disabled) and the **mode bits** .
- **Banked registers:** each interrupt mode has its own copies of some registers. IRQ mode has its own R13 (SP), R14 (LR) and SPSR; **FIQ mode also has its own R8–R12** , so the FIQ routine need not save registers -> very fast.

ARM interrupt response

1. Finish the current instruction; copy **CPSR into SPSR** of the new mode.
2. Change to **IRQ/FIQ mode** , set the **I bit** (and F bit for FIQ) to disable further interrupts.
3. Store the **return address in R14 (LR)** of that mode.
4. Load **PC with the vector address** . Return: SUBS PC, LR, #4 (also restores CPSR from SPSR).

Exception	Vector address
Reset	0x00
Undefined instruction	0x04
Software interrupt (SWI)	0x08
Prefetch abort	0x0C
Data abort	0x10
IRQ	0x18
FIQ	0x1C

- A **vectored interrupt controller** is added when many devices share IRQ.

3.2 Intel IA-32 (Pentium)

- Supports **both** memory-mapped I/O and **isolated I/O** . Isolated I/O uses **IN** and **OUT** instructions with a **16-bit port address** (up to 64K ports); the port address is an

immediate value or in register **DX** .

- **Interrupt pins: INTR** (maskable, controlled by the **IF** flag; **STI** sets IF, **CLI** clears IF) and **NMI** (non-maskable).
- **256 interrupt types** (vectors 0-255): hardware interrupts, software interrupts (**INT n**) and exceptions (0 = divide error, 2 = NMI, 3 = breakpoint ...).
- The type number indexes the **Interrupt Descriptor Table (IDT)** (4-byte entries in real mode, 8-byte descriptors in protected mode) that gives the ISR address.
- On an interrupt the processor **pushes EFLAGS, CS and EIP** on the stack and clears IF. **IRET** pops them and returns.
- A **programmable interrupt controller (8259A)** handles up to 8 devices (cascade for 15). It sends the vector number to the CPU during **INTA** and handles priorities.

Feature	ARM	Intel IA-32
I/O method	Memory-mapped only	Isolated (IN/OUT) and memory-mapped
Interrupt inputs	IRQ and FIQ	INTR and NMI
Disable flag	I and F bits in CPSR	IF flag in EFLAGS
Vector table	Fixed addresses (0x00-0x1C)	256 entries (IDT)
Return	SUBS PC, LR, #4	IRET

4. Direct Memory Access (DMA)

In program-controlled and interrupt-driven I/O the processor moves **every word** , which is too slow for large blocks (disk, network). **DMA** lets a special **DMA controller** transfer a whole block **directly between the I/O device and main memory without the processor** .

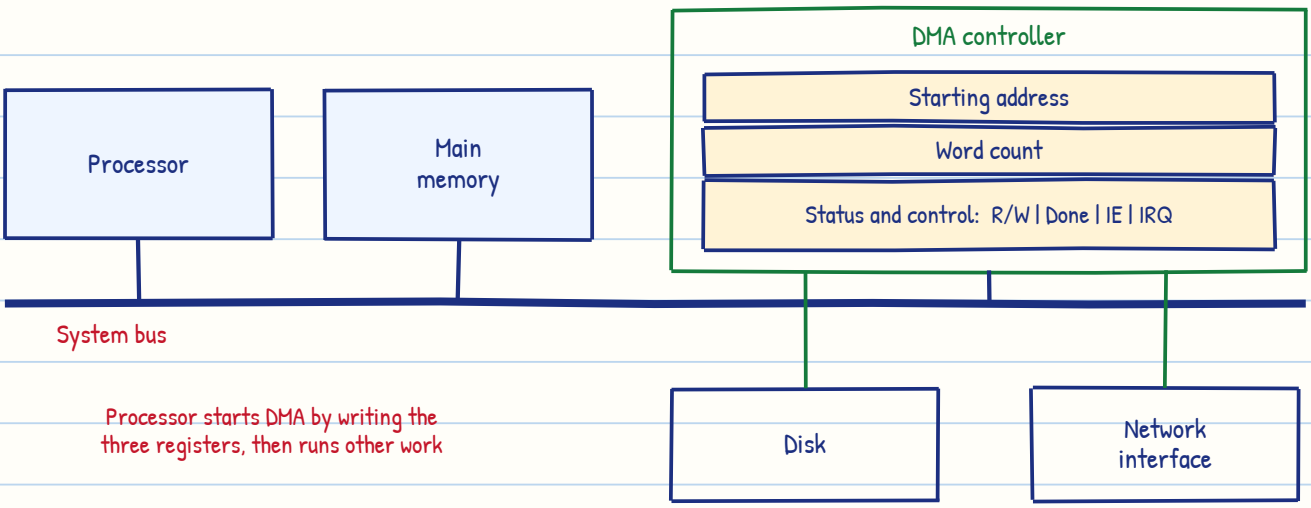


Fig 5: DMA controller in a computer system

DMA controller registers

- **Starting address register** - memory address where the block starts.
- **Word count register** - number of words still to transfer.
- **Status and control register** - contains **R/W bit** (direction of transfer), **Done flag** (set when the block is complete), **IE** bit (interrupt enable) and **IRQ** bit (interrupt request).

Steps of a DMA transfer

1. Processor (through the OS) **loads the three registers** and starts the DMA transfer.
2. The processor continues with **another program** ; the program that wanted I/O waits.
3. DMA controller transfers words between device and memory, **incrementing the address and decrementing the word count** for each word.
4. When the count reaches 0, it sets **Done** and, if **IE = 1** , sends an **interrupt** to the processor. The OS then resumes the waiting program.

Memory access sharing

- Processor and DMA both need the memory bus. The **DMA controller gets priority** because the device must not lose data.
- **Cycle stealing:** DMA takes **one memory cycle at a time** and gives the bus back - processor slows slightly.
- **Block (burst) mode:** DMA keeps the bus for the **whole block** - fast for the device, but the processor is blocked meanwhile.
- **Transparent mode:** DMA uses the bus only when the processor is not using it - no slowdown, but more hardware.

Note: Because DMA writes memory directly, a copy of that data in the cache may become stale (cache coherence). The OS must invalidate or flush the cache for DMA buffers.

Bus arbitration

Only **one device (bus master)** can control the bus at a time. **Arbitration** decides who becomes master. The processor is normally the master; a DMA controller must **request** the bus.

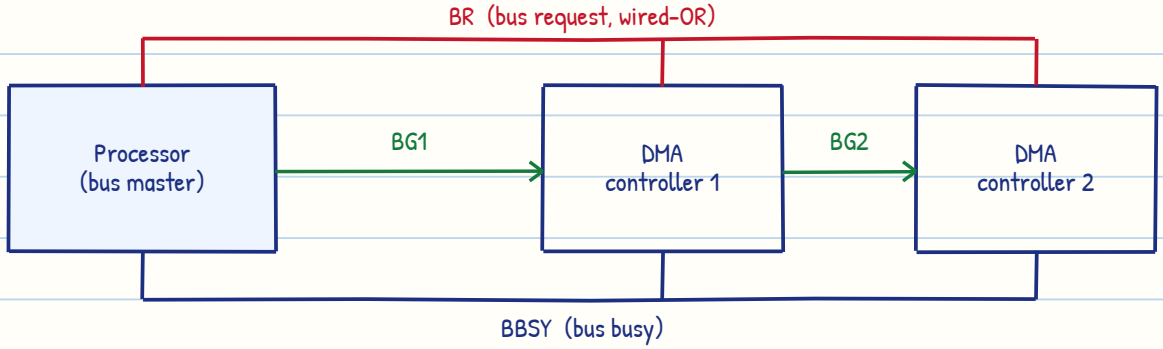


Fig 6: Centralised daisy-chain arbitration

- **BR** (Bus Request) is raised by a device. The processor finishes its bus cycle and sends **BG** (Bus Grant), which passes along the daisy chain. The device that requested keeps BG, raises **BBSY** (busy) and becomes master. It drops BBSY when finished.
- **Centralised arbitration:** one **arbiter** (usually in the processor) grants the bus - by **daisy chain** or **independent BR/BG pair per device** (faster, more wires).
- **Distributed arbitration:** no central arbiter. Each device has a **4-bit ID** and puts it on shared **open-collector lines** (wired-OR). Each device compares the bus value with its ID; a device that sees a higher ID stops driving its lower bits. Finally the bus shows the **highest ID, which wins** . Example: IDs 0101 (5) and 0110 (6) -> bus = 0110, device 6 wins.

	Polling	Interrupt	DMA
Who moves data	Processor	Processor	DMA controller
Processor busy?	Fully (waiting)	Only during ISR	Only at start and end
Best for	Simple, fast devices	Slow, occasional devices	Large blocks (disk, network)

5. Buses

A **bus** is a set of wires that carries **address, data and control** signals between the processor, memory and I/O devices. The **bus protocol** is the set of rules for timing and signals so that the **master** (initiator) and **slave** (target) understand each other.

- **Bus master** starts a transfer (processor or DMA); the addressed **slave** responds.
- **Bandwidth** = bus width x clock rate. Example: 64-bit bus at 100 MHz = 8 bytes x 100 M = **800 MB/s** .
- Two timing methods: **synchronous** (common clock) and **asynchronous** (handshake).

5.1 Synchronous bus

All devices use a **common bus clock** . Every action happens at a fixed clock edge.

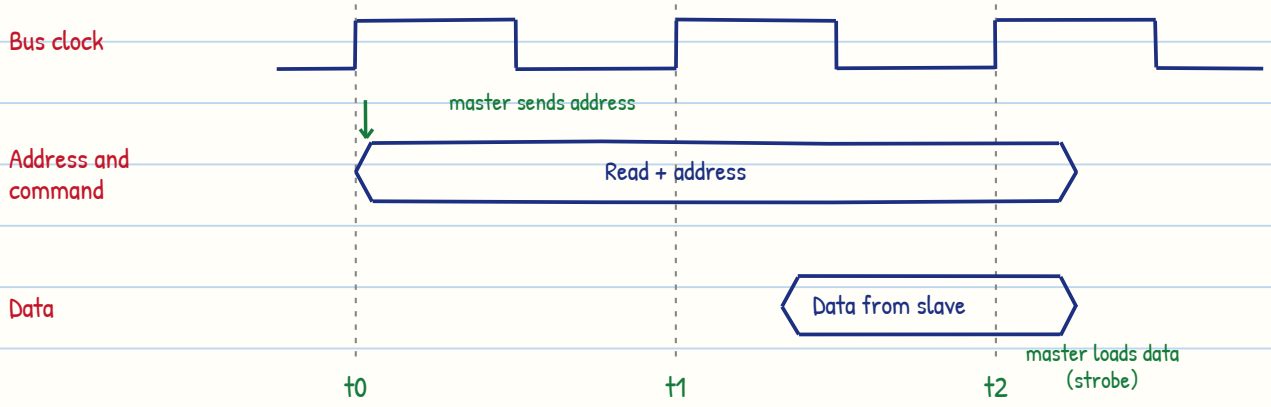


Fig 7: Timing of an input (read) transfer on a synchronous bus

- **t0:** master puts the **address and the Read command** on the bus.
- **t1:** the addressed slave decodes the address and **places the data** on the data lines (after its access time).
- **t2:** master **loads (strobes) the data** into its register and then removes address and command.
- The clock period must be longer than the **maximum propagation delay + set-up time** , so the clock is set for the **slowest** device.
- **Multiple-cycle transfer:** if the slave is slow, it keeps a **Slave-ready** signal low and the master waits for extra clock cycles - so slow and fast devices can share one bus.

5.2 Asynchronous bus

No common clock. The master and slave use two lines, **Master-ready** and **Slave-ready** , in a **full handshake** .

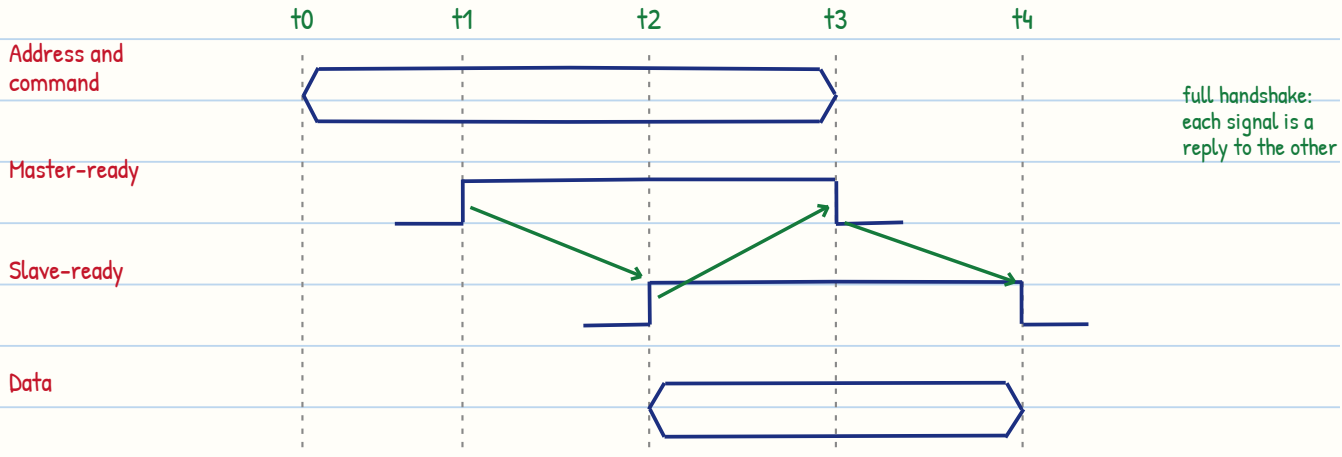


Fig 8: Handshake for an input (read) transfer

1. **t0:** master puts address and command on the bus.
2. **t1:** after a small delay (skew), master asserts **Master-ready** - "address is valid".
3. **t2:** the selected slave performs the operation, puts the data on the bus and asserts **Slave-ready** .
4. **t3:** master loads the data, removes address/command and drops Master-ready.
5. **t4:** slave removes the data and drops Slave-ready. The bus is free.

Synchronous bus	Asynchronous bus
Common clock controls timing	Handshake signals control timing
Simple, fast interface logic	More complex circuits
Speed fixed by the slowest device	Speed adapts to each device
Poor for devices of very different speed	Handles devices of widely different speed
Clock skew limits bus length	Handshake delay slows each transfer

- Large systems use a **bus hierarchy** : a fast processor bus and memory bus, joined by **bridges** to slower I/O buses. Wide buses and fast clocks give higher bandwidth.

6. Interface Circuits

An interface circuit connects an I/O device to the bus. It has **data registers, status flags, an address decoder** and control logic. Interfaces are of two kinds: **parallel port** and **serial port** .

6.1 Parallel interface

All bits of a data item (e.g. 8 bits) are transferred **at once** on separate lines. Fast, but needs many wires - good for short distances (printer, keyboard).

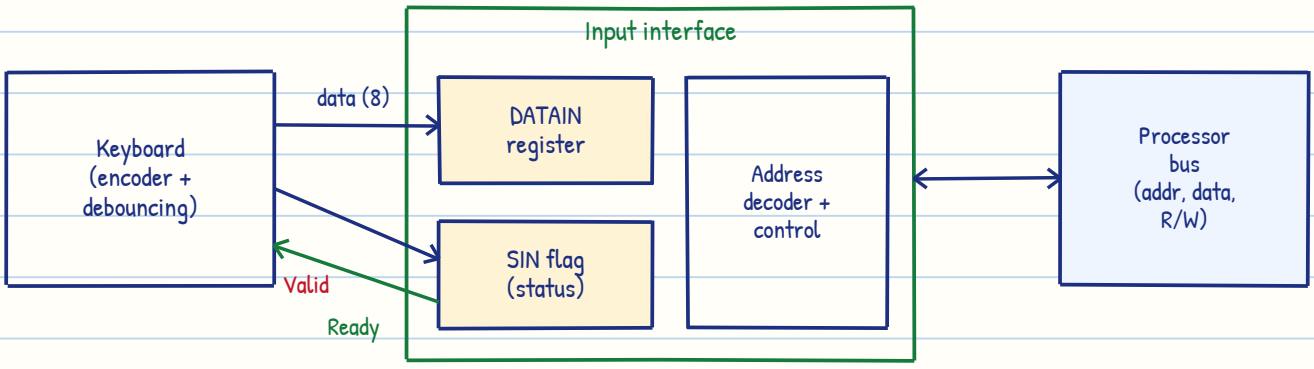


Fig 9: Input interface for a keyboard

- **Keyboard (input):** when a key is pressed the encoder produces the code and a **Valid** signal. The interface loads the code in **DATAIN** and sets **SIN = 1** . The processor reads DATAIN (which clears SIN) and the interface sends **Ready** back, telling the keyboard to

send the next code.

- **Printer (output):** the processor writes **DATAOUT** ; **SOUT** is cleared and the interface sends **Valid** to the printer. When the printer is **idle** again it replies and the interface sets **SOUT = 1** .
- **General parallel port:** each of its 8 lines can be input or output according to a **Data Direction Register (DDR)** , and two **handshake control lines (C1, C2)** can be set up for different modes and can raise interrupts.

6.2 Serial interface

Data is sent **one bit at a time** over a single line. It needs fewer wires, so it suits **long distances** (modem, mouse, network, USB).

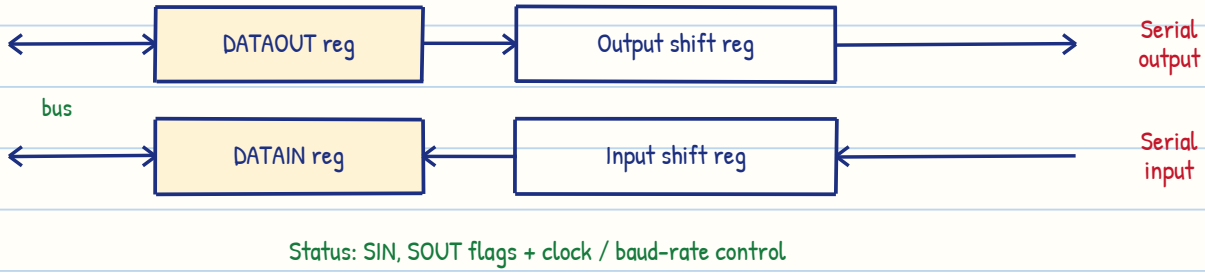


Fig 10: Serial interface and an asynchronous character

- Output: the processor writes a word into **DATAOUT** ; it is copied to the **output shift register** , which shifts the bits out one by one. Input: bits shift into the **input shift register** , then the whole word moves to **DATAIN** and **SIN** is set.
- **Asynchronous serial:** each character has a **start bit (0)** , 5-8 **data bits** , optional **parity bit** and **1 or 2 stop bits (1)** . The idle line is 1. The receiver detects the start bit and samples every bit in the **middle** of its time using a clock **16 times faster** than the bit rate.
- **Baud rate** = signalling speed. Example: 9600 baud with 8N1 (10 bits/char) -> **960 characters/s** . The chip that does this is a **UART** .
- **Synchronous serial:** no start/stop bit for each character; a block of characters is sent with **synchronisation characters** and the receiver recovers the clock - more efficient for big blocks.

Parallel	Serial
Many bits at a time	One bit at a time
Many wires, short distance	Few wires, long distance
Fast for short links; cross-talk problems	Very high speed possible with good signalling
e.g. printer port, internal buses	e.g. USB, Ethernet, SATA, PCI Express

7. Standard I/O Interfaces

A **standard** lets devices from different manufacturers work with any computer that follows it. I/O devices can be connected in three ways: on the **processor bus** (fast, but processor-specific), on a **backplane bus** (standard, processor independent - **PCI**), or via an **I/O bus** (**SCSI, USB**).

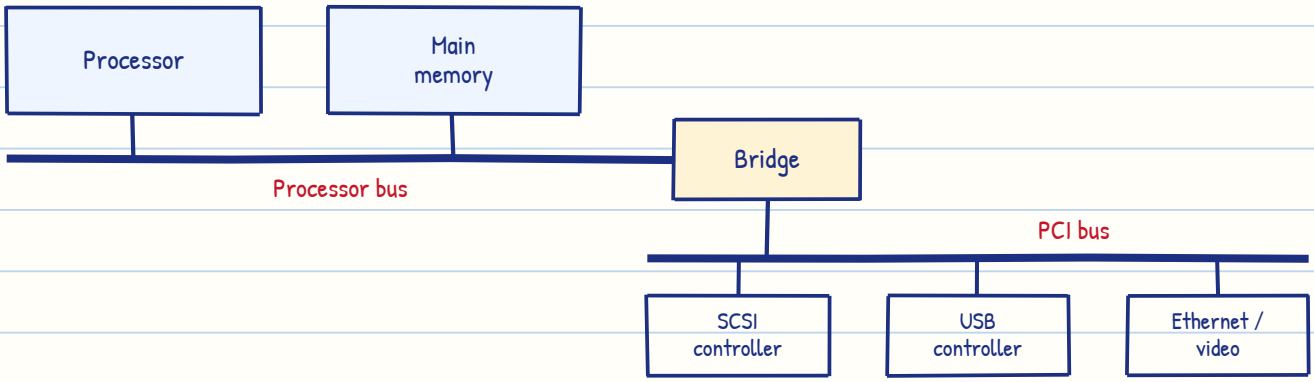


Fig 11: Bus hierarchy with a PCI bridge

7.1 PCI (Peripheral Component Interconnect)

- Developed by **Intel (about 1992)** ; processor independent. A **bridge** connects the processor bus to the PCI bus, so changing the processor does not change the PCI devices.
- Width **32 bits** (extendable to 64) at **33 MHz** (or 66 MHz): 32 x 33 MHz = **132 MB/s** ; 64-bit at 66 MHz = **528 MB/s** .
- **Plug and play:** every device has a small **configuration space/ROM** holding its type, memory and interrupt needs; the system configures it automatically at start-up.
- Address and data lines are **multiplexed** to save pins; supports **burst transfers** ; has a **central arbiter** . Devices act as **initiators or targets** .
- **PCI Express** is the later serial, point-to-point version using **lanes** .

7.2 SCSI (Small Computer System Interface)

- An **ANSI standard** bus for disks, tapes, CD drives and scanners.
- **Narrow SCSI** is 8 bits wide (up to **8 devices** , IDs 0-7 including the controller); **wide**

SCSI is 16 bits (up to **16 devices**).

- Devices are connected in a **daisy chain** (one cable, **terminators** at both ends). Each device has a **unique ID** ; a device acts as **initiator** (controller) or **target** (disk).
- A transfer goes through **phases** : arbitration -> selection -> information transfer (command, data, status, message) -> bus free. Both asynchronous and synchronous modes exist. Fast versions reach **320 MB/s** (Ultra320).

7.3 USB (Universal Serial Bus)

- Designed by a group of companies (Intel, Microsoft, HP, NEC, Philips and others) as a **simple, low-cost, plug-and-play, hot-pluggable** way to connect keyboards, mice, printers, cameras, drives etc.

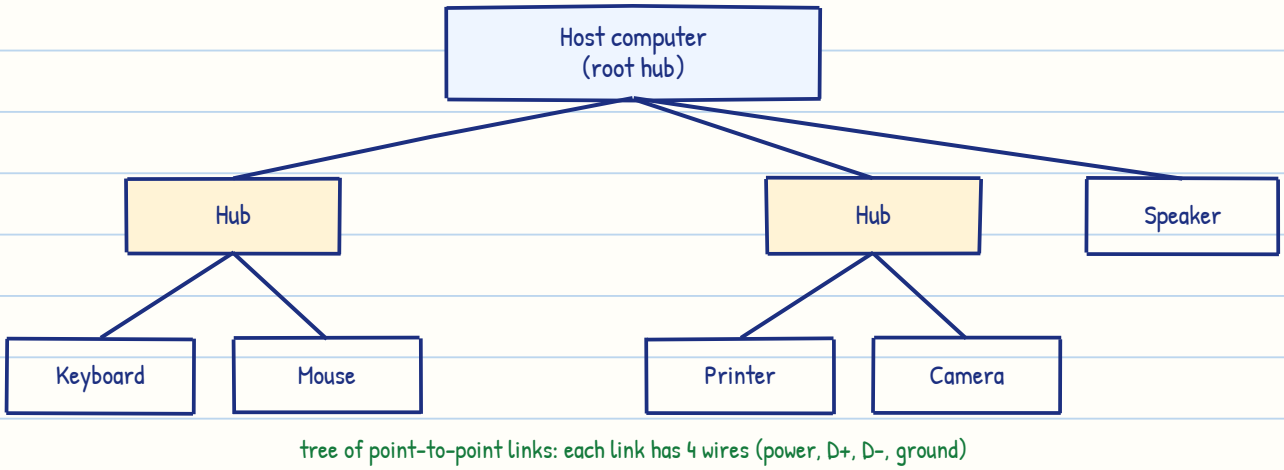


Fig 12: USB tree structure with hubs

- **Tree of hubs:** the **root hub** is in the host computer; hubs have more ports. Up to **127 devices** can be attached. Each device gets a **7-bit address** when plugged in (**enumeration**).
- Cable: **4 wires - +5 V power, D+, D- (differential data pair), ground** . The bus can **power small devices** .
- Speeds: **Low 1.5 Mbps** (mouse, keyboard), **Full 12 Mbps** , **High 480 Mbps** (USB 2.0), **SuperSpeed 5 Gbps** (USB 3.0).
- **Communication:** the host **polls** ; all transfers use **packets - token** (IN, OUT, SETUP), **data** and **handshake** (ACK, NAK, STALL). Data is coded with **NRZI + bit stuffing** for clock recovery. Time is split into **1 ms frames** .
- **Transfer types: Control** (setup), **Bulk** (printers, disks), **Interrupt** (keyboard/mouse - polled regularly), **Isochronous** (audio/video - fixed rate, no retry).
- **Plug and play:** when a device is connected, the hub tells the host; the host reads its descriptor, assigns an address and loads the right driver.

	PCI	SCSI	USB
Type	Backplane (internal) bus	I/O bus, daisy chain	Serial I/O bus, tree
Data path	32/64-bit parallel	8/16-bit parallel	1 bit (differential pair)
Devices	Add-in cards	8 or 16 (disks, tape)	Up to 127 (many kinds)
Plug and play	Yes (configuration)	Limited	Yes, hot-pluggable

Note: Other standards you may meet: FireWire (IEEE 1394), SATA (serial disk interface), Thunderbolt, PCI Express.

Quick Revision and Important Questions

- Access: memory-mapped (normal instructions) vs isolated (IN/OUT). Polling wastes CPU time -> use interrupts / DMA.
- Interrupt: finish instruction -> save PC and PS -> ISR -> return. Vectored = fast; daisy chain = priority; exceptions = internal.
- DMA registers: address, word count, status/control. Modes: cycle stealing, burst, transparent. Arbitration: centralised or distributed.
- Synchronous bus = clock; asynchronous = Master-ready / Slave-ready handshake. Serial frame = start + data + stop.
- PCI (backplane, plug and play), SCSI (daisy chain, 8/16 devices), USB (tree, hubs, 127 devices).

Likely exam questions

1. Explain memory-mapped and isolated I/O. Describe the I/O interface with a diagram.
2. Explain program-controlled I/O with an example program.
3. Explain interrupt handling. Discuss vectored interrupts, nesting and priority (daisy chain).
4. What are exceptions? Give examples.
5. Explain interrupt handling in ARM and Intel processors.
6. Explain DMA with a block diagram. Describe the DMA registers and transfer modes.
7. Explain bus arbitration (centralised and distributed).
8. Compare synchronous and asynchronous buses with timing diagrams.
9. Explain parallel and serial interface circuits.
10. Write short notes on PCI, SCSI and USB.